

四、核心解析

1. 第一题：任务划分推导

8 个任务的算力需求总和为 $1600 \times 3 = 4800$ 。根据已知条件：

- 各任务算力需求分别为：任务1 (1000)、任务2 (800)、任务3 (500)、任务4 (600)、任务5 (300)、任务6 (700)、任务7 (500)、任务8 (400)。
- 节点 1 已分配 [1, 4]，算力需求 $1000 + 600 = 1600$ (已满载)。
- 剩余任务算力需求组合需将剩余任务分为两组，使得每组和为 1600：
 - **节点 2**： $800 + 500 + 300 = 1600 \rightarrow$ 任务 [2, 3, 5]
 - **节点 3**： $700 + 500 + 400 = 1600 \rightarrow$ 任务 [6, 7, 8]

2. 第二题：直接插入排序（降序）

- t 为当前待插入任务的算力需求。
- 当满足 $j \geq 0$ 且前面的元素 $\text{task}[j][1]$ 的算力需求比 t 小 ($\text{task}[j][1] < t$) 时，原元素向后移动一位： $\text{task}[j + 1] = \text{task}[j]$ 。
- 组合边界与条件填入： $j \geq 0$ and $\text{task}[j][1] < t$ 。

3. 第三题：动态规划（0-1背包）与回溯撤销

① $p > 0$ —— 逆向追溯路径

- $a[m] == m$ 表示找到了刚好凑满 m 容量的组合， p 初始赋值为 m 。
- 利用 $\text{lnk}[p]$ 记录的“上一步选择的任务索引”，倒推已被选中的任务。
- 每推导出一个任务，容量就扣减 $p -= \text{tasks}[j][1]$ ，直到容量 p 扣减完毕至 0 止，因此循环条件为 $p > 0$ 。

关键点解释： 为何不能写 $p = \text{lnk}[p]$ ？

- p 代表的是**剩余容量/算力**（如 1600, 1000, 0）。
- $\text{lnk}[p]$ 代表的是**任务下标/索引**（如 0, 3）。
- 两者物理意义完全不同，扣减剩余容量必须使用 $p -= \text{tasks}[j][1]$ 。

② $\text{found} = \text{findflops}()$ —— 调用函数分配算力

- $\text{while len(q)} \neq \text{cnt}$ ：为主循环，每次尝试为当前节点分配一组满载任务。
- 使用 found 接收 $\text{findflops}()$ 的返回值（True 表示找到满载分配方案，False 表示无解）。

③ `flag[tmp[1]] = False` —— 状态回溯撤销

- 当后续节点分配无解时，触发状态回溯：`tmp = q.pop(0)` 弹出一项分配记录，`tmp[0]` 为节点号，`tmp[1]` 为任务索引。
- 需要撤销该任务的已占用状态，将其还原为可分配状态，故将标志位重置：`flag[tmp[1]] = False`。

五、重点知识扩展：0-1 背包为什么要倒序遍历容量？

在 `findflops()` 中，遍历算力容量的循环代码为：

Python

```
for j in range(m, t - 1, -1):
```

核心原因

为了防止同一个任务在同一轮循环中被“重复挑选”。

对比分析

- **倒序遍历（正确）**：计算容量 j 时，参考的较小容量 $j - t$ 尚未在当前轮更新过，保留的是上一步未选择该任务时的状态，确保当前任务**只使用 1 次（0-1背包）**。
- **顺序遍历（错误）**：计算较小容量（如 $j = 600$ ）时，更新了数据；再计算更大容量（如 $j = 1200$ ）时，参考的 $j - t$ 已经被写入了刚才更新的数据，导致同一个任务被**多次选择（退化为完全背包）**。